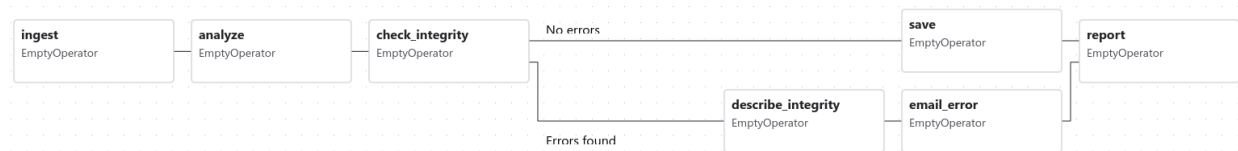


# APACHE AIRFLOW

Airflow is a platform that lets you build and run *workflows*. A workflow is represented as a DAG (a Directed Acyclic Graph), and contains individual pieces of work called Tasks, arranged with dependencies and data flows taken into account.



A DAG specifies the dependencies between tasks, which defines the order in which to execute the tasks. Tasks describe what to do, be it fetching data, running analysis, triggering other systems, or more.

Airflow itself is agnostic to what you're running - it will happily orchestrate and run anything, either with high-level support from one of our providers, or directly as a command using the shell or Python Operators.

## Airflow components

Airflow's architecture consists of multiple components. The following sections describe each component's function and whether they're required for a bare-minimum Airflow installation, or an optional component to achieve better Airflow extensibility, performance, and scalability.

### Required components

A minimal Airflow installation consists of the following components:

# APACHE AIRFLOW

- A scheduler, which handles both triggering scheduled workflows, and submitting Tasks to the executor to run. The executor, is a configuration property of the *scheduler*, not a separate component and runs within the scheduler process. There are several executors available out of the box, and you can also write your own.
- A *dag processor*, which parses DAG files and serializes them into the *metadata database*. More about processing DAG files can be found in DAG File Processing
- A *webserver*, which presents a handy user interface to inspect, trigger and debug the behaviour of dags and tasks.
- A folder of *DAG files*, which is read by the *scheduler* to figure out what tasks to run and when to run them.
- A *metadata database*, which Airflow components use to store state of workflows and tasks. Setting up a metadata database is described in Set up a Database Backend and is required for Airflow to work.

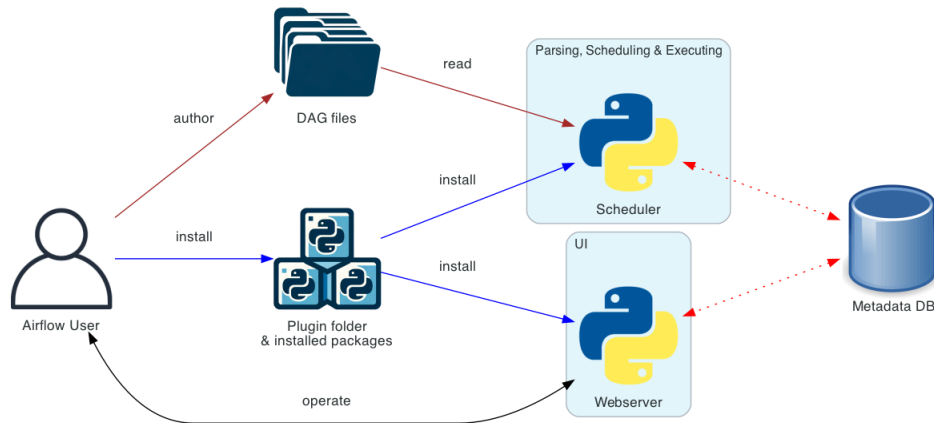
## Optional components

Some Airflow components are optional and can enable better extensibility, scalability, and performance in your Airflow:

- Optional *worker*, which executes the tasks given to it by the scheduler. In the basic installation worker might be part of the scheduler not a separate component. It can be run as a long running process in the CeleryExecutor, or as a POD in the KubernetesExecutor.
- Optional *triggerer*, which executes deferred tasks in an asyncio event loop. In basic installation where deferred tasks are not used, a triggerer is not necessary. More about deferring tasks can be found in Deferrable Operators & Triggers.
- Optional folder of *plugins*. Plugins are a way to extend Airflow's functionality (similar to installed packages). Plugins are read by the *scheduler*, *dag processor*, *triggerer* and *webserver*. More about plugins can be found in Plugins.
- 

## Architecture Diagrams

# APACHE AIRFLOW



The diagrams below show different ways to deploy Airflow - gradually from the simple “one machine” and single person deployment, to a more complex deployment with separate components, separate user roles and finally with more isolated security perimeters.

The meaning of the different connection types in the diagrams below is as follows:

- brown solid lines represent *DAG files* submission and synchronization
- blue solid lines represent deploying and accessing *installed packages* and *plugins*
- black dashed lines represent control flow of workers by the *scheduler* (via executor)
- black solid lines represent accessing the UI to manage execution of the workflows
- red dashed lines represent accessing the *metadata database* by all components

## Basic Airflow deployment

# APACHE AIRFLOW

This is the simplest deployment of Airflow, usually operated and managed on a single machine. Such a deployment usually uses the LocalExecutor, where the *scheduler* and the *workers* are in the same Python process and the *DAG files* are read directly from the local filesystem by the *scheduler*. The *webserver* runs on the same machine as the *scheduler*. There is no *triggerer* component, which means that task deferral is not possible.

Such an installation typically does not separate user roles - deployment, configuration, operation, authoring and maintenance are all done by the same person and there are no security perimeters between the components.

If you want to run Airflow on a single machine in a simple single-machine setup, you can skip the more complex diagrams below and go straight to the Workloads section.

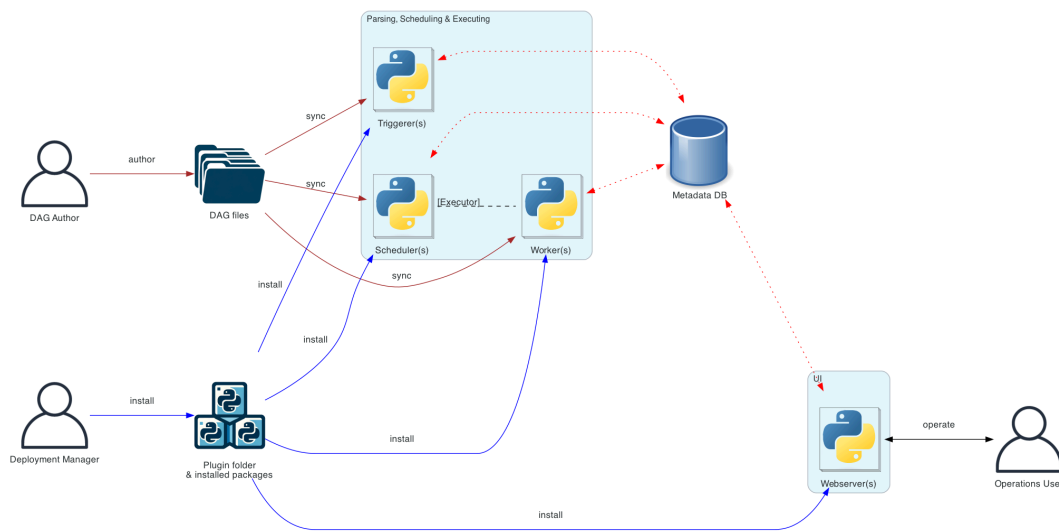
## Distributed Airflow architecture

This is the architecture of Airflow where components of Airflow are distributed among multiple machines and where various roles of users are introduced - *Deployment Manager*, DAG author, Operations User. You can read more about those various roles in the Airflow Security Model.

In the case of a distributed deployment, it is important to consider the security aspects of the components. The *webserver* does not have access to the *DAG files* directly. The code in the Code tab of the UI is read from the *metadata database*. The *webserver* cannot execute any code submitted by the DAG author. It can only execute code that is installed as an *installed package* or *plugin* by the Deployment Manager. The Operations User only has access to the UI and can only trigger dags and tasks, but cannot author dags.

# APACHE AIRFLOW

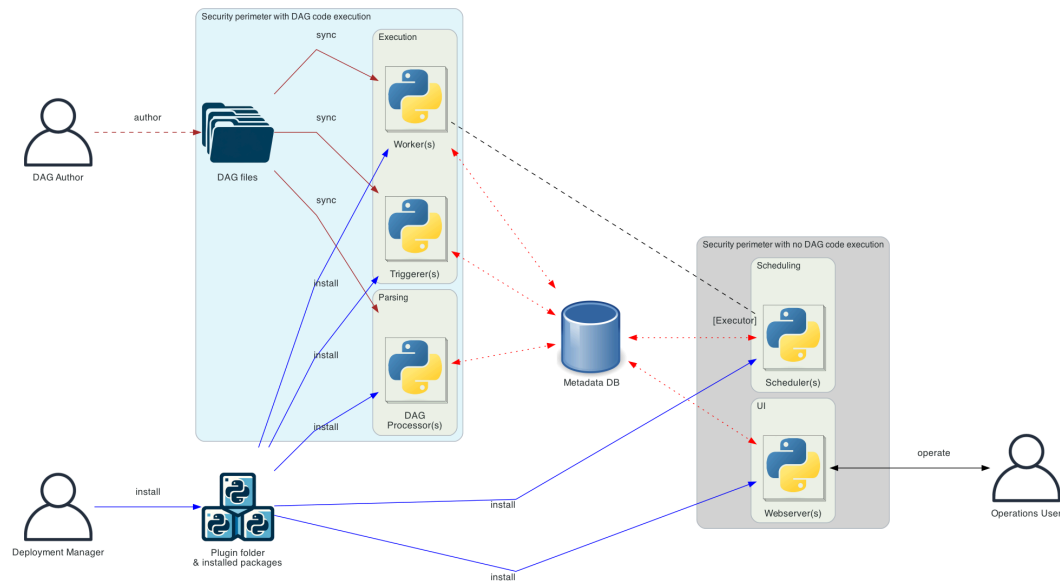
The *DAG files* need to be synchronized between all the components that use them - *scheduler*, *triggerer* and *workers*. The *DAG files* can be synchronized by various mechanisms - typical ways how dags can be synchronized are described in Manage dag files of our Helm Chart documentation. Helm chart is one of the ways how to deploy Airflow in K8S cluster.



## Separate DAG processing architecture

In a more complex installation where security and isolation are important, you'll also see the standalone *dag processor* component that allows to separate *scheduler* from accessing *DAG files*. This is suitable if the deployment focus is on isolation between parsed tasks. While Airflow does not yet support full multi-tenant features, it can be used to make sure that DAG author provided code is never executed in the context of the scheduler.

# APACHE AIRFLOW



When DAG file is changed there can be cases where the scheduler and the worker will see different versions of the DAG until both components catch up. You can avoid the issue by making sure dag is deactivated during deployment and reactivate once finished. If needed, the cadence of sync and scan of DAG folder can be configured. Please make sure you really know what you are doing if you change the configurations.

## Dags

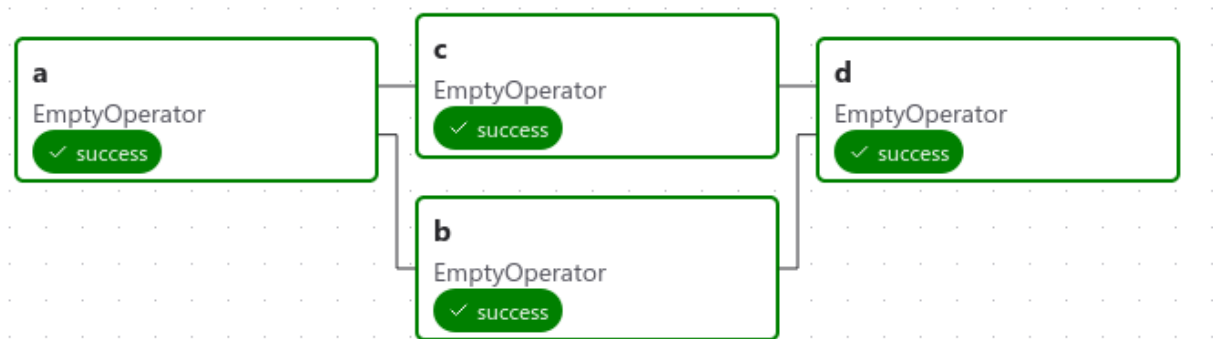
A DAG is a model that encapsulates everything needed to execute a workflow. Some DAG attributes include the following:

- **Schedule:** When the workflow should run.

# APACHE AIRFLOW

- Tasks: tasks are discrete units of work that are run on workers.
- Task Dependencies: The order and conditions under which tasks execute.
- Callbacks: Actions to take when the entire workflow completes.
- Additional Parameters: And many other operational details.

Here's a basic example DAG:



It defines four Tasks - A, B, C, and D - and dictates the order in which they have to run, and which tasks depend on what others. It will also say how often to run the DAG - maybe “every 5 minutes starting tomorrow”, or “every day since January 1st, 2020”.

The DAG itself doesn't care about *what* is happening inside the tasks; it is merely concerned with *how* to execute them - the order to run them in, how many times to retry them, if they have timeouts, and so on.

The term “DAG” comes from the mathematical concept “directed acyclic graph”, but the meaning in Airflow has evolved well beyond just the literal data structure associated with the mathematical DAG concept.

# APACHE AIRFLOW